

بسمه تعالی

Final Project- Deep Neural Networks

Supervisor:

Prof. Johari Majd

Student:

Morteza Bigdeli - 40261662001

Tarbiat Modares University

Spring 2024

Table of Contents

1.AUTO ENCODER.....	4
1-1- Introduction.....	5
1-2- Problem statement.....	5
1-3-Methodology.....	6
1-4-Model Architecture and training process	6
1-5-Results.....	8
2.VARIATIONAL AUTO ENCODER.....	16
2-1-Introduction.....	16
2-2-Problem statement.....	16
2-3-Methodology.....	17
2-4-Model Architecture.....	17
2-5-Results.....	18

Table of Figures

Figure 1. Autoencoder Model	4
Figure 2. losses in phases for basic architecture.....	9
Figure 3. Latent space with basic architecture.....	9
Figure 4. Test result with basic architecture	10
Figure 5. losses for each phase with added neurons	10
Figure 6. Test result with added neurons architecture.....	10
Figure 7. Latent space with added neurons architecture	11
Figure 8. losses for each phase with added neurons and layers.....	12
Figure 9. Test result with added neurons and layers	12
Figure 10. Latent space with added neurons and layers	13
Figure 11. losses for each phase, added more neurons and layers	14
Figure 12. Test result with added more neurons and layers	14
Figure 13. Latent space with added more neurons and layers	15
Figure 14. VAE losses for each phase, basic architecture	18
Figure 15. VAE latent space with basic architecture.....	19
Figure 16. VAE test result with basic architecture	19
Figure 17. VAE losses for each phase, added neurons.....	22
Figure 18. VAE latent space, added neurons.....	22
Figure 19. VAE test data, added neurons	23
Figure 20. VAE losses for each phase, added neurons with noise	23
Figure 21. VAE latent space, added neurons and noise.....	24
Figure 22. VAE test data, added neurons and noise	24
Figure 23. VAE losses for each phase, added neurons and layers with noise	25
Figure 24. VAE latent space, added neurons, layers and noise	26
Figure 25. VAE test data, added neurons, layers and noise.....	27

1. Auto Encoder

1. Train an autoencoder for both denoising and classification of MNIST simultaneously.

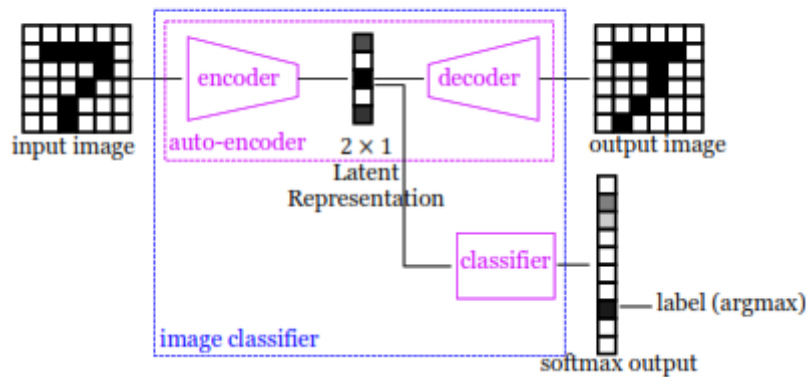


Figure 1. Autoencoder Model

Add 10% salt and pepper, and 10% dropout noise (see lecture5, slides 50-51) to input images.

Use the following encoder architecture (You may modify some parameters if you need to):

Input Image $\rightarrow$ Conv(1,8,4,2,0) $\rightarrow$ Dropout(0.1) $\rightarrow$
ReLU $\rightarrow$ Conv(8,6,3,2,0) $\rightarrow$ Dropout(0.1) $\rightarrow$ ReLU $\rightarrow$
Conv(6,4,3,3,0) $\rightarrow$ Dropout(0.1) $\rightarrow$ ReLU
Conv(4,2,2,1,0) $\rightarrow$ Dropout(0.1) $\rightarrow$ ReLU

Classification part:

Encoder output $\rightarrow$ Linear (2,10) $\rightarrow$ Softmax.

For the decoder use transposed convolution (again you may change some parameters if you need to):

Encoder Output $\rightarrow$ TransposedConv(2,4,2,1,0) $\rightarrow$ ReLU
 $\rightarrow$ Dropout(0.1) $\rightarrow$ TransposedConv(4,6,3,3,0) $\rightarrow$ ReLU
 $\rightarrow$ Dropout(0.1) $\rightarrow$ TransposedConv(6,8,3,2,0) $\rightarrow$ ReLU
 $\rightarrow$ Dropout(0.1) $\rightarrow$ TransposedConv(8,1,4,2,0) $\rightarrow$ Sigmoid

Since we also want to train autoencoder with classification capabilities, there will be two different losses:

- a) Classification loss $\mathcal{L}_c$: Classic cross entropy should do the trick. This loss will take compressed latent representation (C dimensional) and target labels to calculate negative log likelihood loss. This loss will force encoder to output compressed representation such that it aligns well with the target class.
- b) Reconstruction loss $\mathcal{L}_r$: Current autoencoder MSE reconstruction loss. This will force network to output an image as close as possible to given image by taking the compressed representation.
 - i) First, only train encoder to decrease classification loss $\mathcal{L}_c$.
 - ii) Then freeze encoder parameters, and train decoder to decrease reconstruction loss $\mathcal{L}_r$.
 - iii) Finally, train both together and decrease the following loss:

1-1- Introduction

This report outlines the implementation of an autoencoder model applied to the MNIST dataset. The autoencoder is designed to perform both image reconstruction and classification tasks. The report details the architecture of the model, training procedure, hyperparameters used, and results obtained from the training and evaluation phases.

1-2- Problem statement

1. **Train an Autoencoder:**
 - Develop an autoencoder model that can denoise and classify MNIST images.
 - Add 10% salt and pepper noise and 10% dropout noise to the input images.
2. **Network Architecture:**
 - **Encoder:** Multiple convolutional layers with dropout and ReLU activations.
 - **Classifier:** Linear layers with Softmax activation.
 - **Decoder:** Transposed convolutional layers with dropout and ReLU activations.
3. **Loss Functions:**
 - **Classification Loss ($\mathcal{L}_c$):** Cross-entropy loss.

- **Reconstruction Loss ($\mathcal{L}_r$):** Mean Squared Error (MSE).
- 4. **Training Procedure:**
 - Train the encoder for classification.
 - Freeze encoder parameters and train the decoder for reconstruction.
 - Train both together with a combined loss function.

1-3- Methodology

Data Preparation:

The MNIST dataset, consisting of 60,000 training images and 10,000 test images of handwritten digits (0-9), is used. Each image is a 28x28 grayscale pixel image.

Data Augmentation

To enhance the robustness of the model, the following noise is added to the dataset:

Salt and Pepper Noise: 10% of the pixels are randomly set to maximum (white) or minimum (black) values.

Dropout Noise: 10% dropout is applied to the input images to simulate further noise.

1-4- Model Architecture and training process

The autoencoder model consists of three main components: the encoder, the classifier, and the decoder.

Encoder:

The encoder compresses the input image into a lower-dimensional latent representation using a series of convolutional layers. Dropout layers are added to prevent overfitting, and ReLU activation functions introduce non-linearity.

- **Layer 1:**
 - Convolution: 1 input channel, 8 output channels, kernel size of 4, stride of 2, padding of 0.
 - Dropout: 0.1
 - Activation: ReLU
- **Layer 2:**
 - Convolution: 8 input channels, 6 output channels, kernel size of 3, stride of 2, padding of 0.
 - Dropout: 0.1
 - Activation: ReLU
- **Layer 3:**
 - Convolution: 6 input channels, 4 output channels, kernel size of 3, stride of 3, padding of 0.

- Dropout: 0.1
- Activation: ReLU
- **Layer 4:**
 - Convolution: 4 input channels, 2 output channels, kernel size of 2, stride of 1, padding of 0.
 - Dropout: 0.1
 - Activation: ReLU

Classifier:

The classifier transforms the latent representation into class probabilities using fully connected layers and a softmax activation function.

- **Layer 1:**
 - Fully Connected: 2 input features, 10 output features.
 - Activation: ReLU
- **Layer 2:**
 - Fully Connected: 10 input features, 10 output features.
 - Activation: Softmax (dim=1)

Decoder:

The decoder reconstructs the input image from the latent representation using a series of transposed convolutional layers, which expand the dimensions back to the original image size. Dropout layers are again used to prevent overfitting, and ReLU activations introduce non-linearity. A Sigmoid activation at the final layer ensures pixel values are between 0 and 1.

- **Layer 1:**
 - Transposed Convolution: 2 input channels, 4 output channels, kernel size of 2, stride of 1, padding of 0.
 - Activation: ReLU
 - Dropout: 0.1
- **Layer 2:**
 - Transposed Convolution: 4 input channels, 6 output channels, kernel size of 3, stride of 3, padding of 0.
 - Activation: ReLU
 - Dropout: 0.1
- **Layer 3:**
 - Transposed Convolution: 6 input channels, 8 output channels, kernel size of 3, stride of 2, padding of 0.
 - Activation: ReLU
 - Dropout: 0.1
- **Layer 4:**
 - Transposed Convolution: 8 input channels, 1 output channel, kernel size of 4, stride of 2, padding of 0.
 - Activation: Sigmoid

Training Procedure:

The training is conducted in three phases to ensure effective learning of both classification and reconstruction tasks.

Phase 1: Classification Training

1. **Objective:** Train the encoder and classifier to minimize classification loss ($\mathcal{L}_c$).
2. **Loss Function:** CrossEntropyLoss.
3. **Optimizer:** Adam optimizer with a learning rate of $1e-3$.
4. **Procedure:** Train for 10 epochs, optimizing only the encoder and classifier parameters.

Phase 2: Reconstruction Training

1. **Objective:** Train the decoder to minimize reconstruction loss ($\mathcal{L}_r$).
2. **Loss Function:** Mean Squared Error (MSE).
3. **Optimizer:** Adam optimizer with a learning rate of $1e-3$.
4. **Procedure:** Freeze encoder parameters and train the decoder for 10 epochs.

Phase 3: Combined Training

1. **Objective:** Fine-tune the entire autoencoder to optimize a combined loss function.
2. **Loss Function:** Combined loss, $\mathcal{L} = \alpha\mathcal{L}_c + (1 - \alpha)\mathcal{L}_r$, with α set to 0.5.
3. **Optimizer:** Adam optimizer with a learning rate of $1e-3$.
4. **Procedure:** Train both the encoder and decoder for 10 epochs.

Evaluation:

The performance of the trained autoencoder is evaluated based on:

1. **Classification Accuracy:** Accuracy of the classifier on the test set.
2. **Reconstruction Quality:** Visual inspection and mean squared error of the reconstructed images compared to the original images.
3. **Latent Space Visualization:** Generate and visualize output for equally spaced 2-D vectors in the latent space to assess the semantic understanding of the input data.

The results are visualized using plots to illustrate the classification loss, reconstruction loss, and combined loss over training iterations. Reconstructed images and latent space visualizations provide additional qualitative evaluation of the model's performance.

Several architects have been employed which will be reported in next part as results

1-5- Results

The python code for this part has been named as Q1 in the uploaded file.

In first step the basic architecture of the question has been employed for the implementation. Fig (2) shows the losses for the phases, and the Fig (3) shows the latent space which is not clear at all due to the configuration and the added noise. Fig (4) shows the test results, the real images and the digits with pepper and salt added noise. It is obvious that the detections was not well enough.

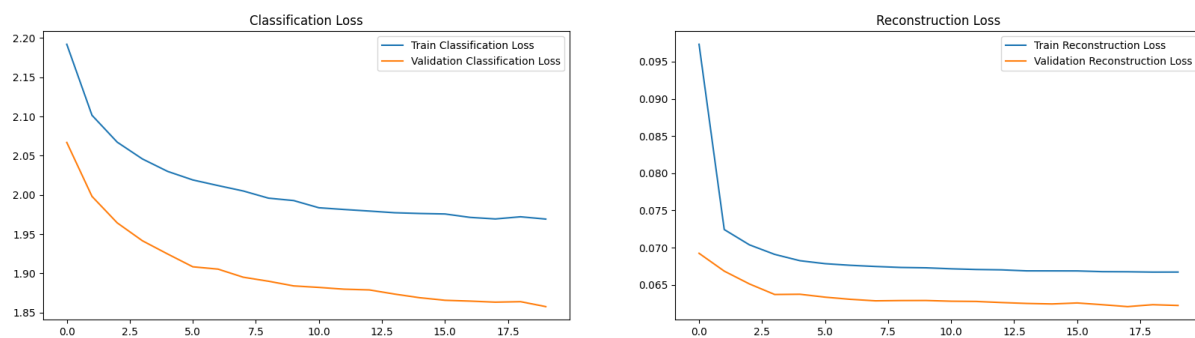


Figure 2. losses in phases for basic architecture

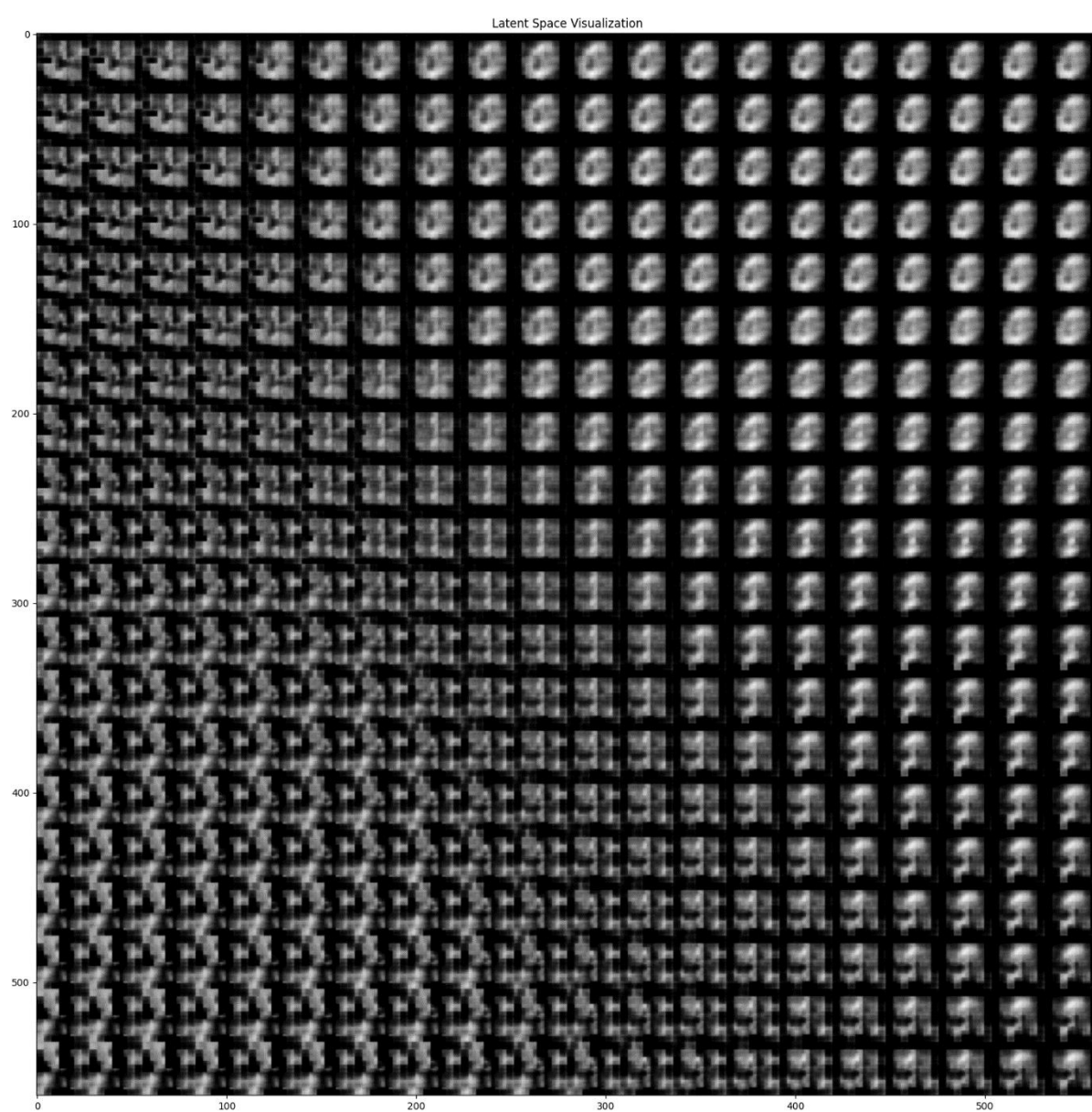


Figure 3. Latent space with basic architecture

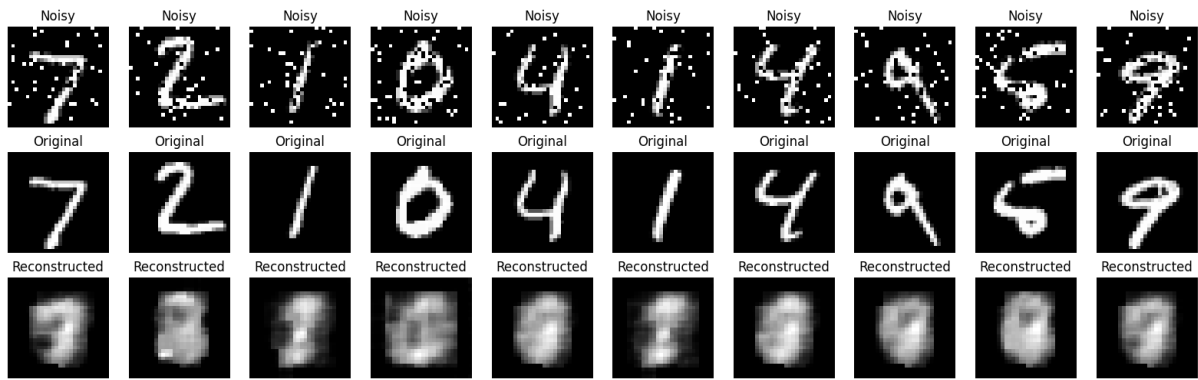


Figure 4. Test result with basic architecture

At next step, I added some neurons to the same layers, still by considering the Fig (5) for losses, Fig (7) for latent space, and Fig (6) for the test data, outputs are not well enough. The detection became better but still the latent space is not what we acquire.

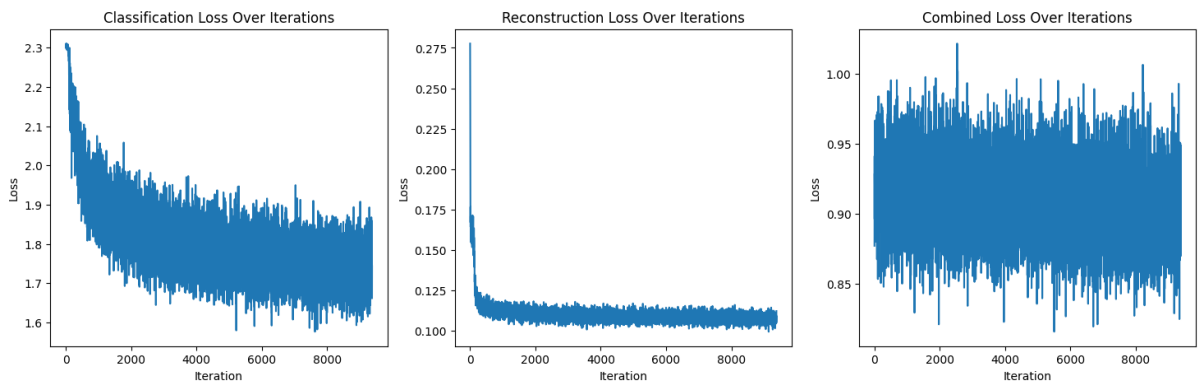


Figure 5. losses for each phase with added neurons

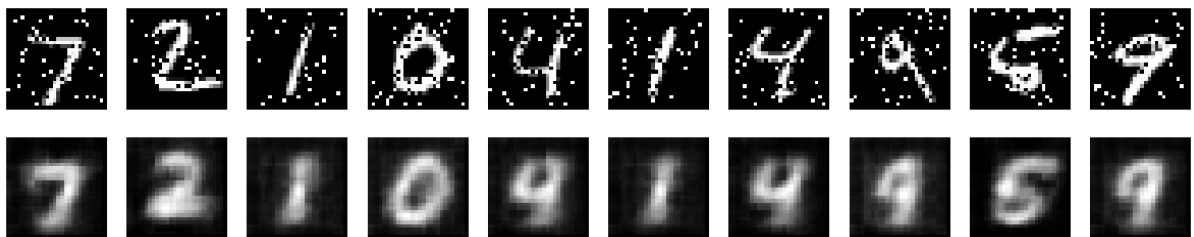


Figure 6. Test result with added neurons architecture

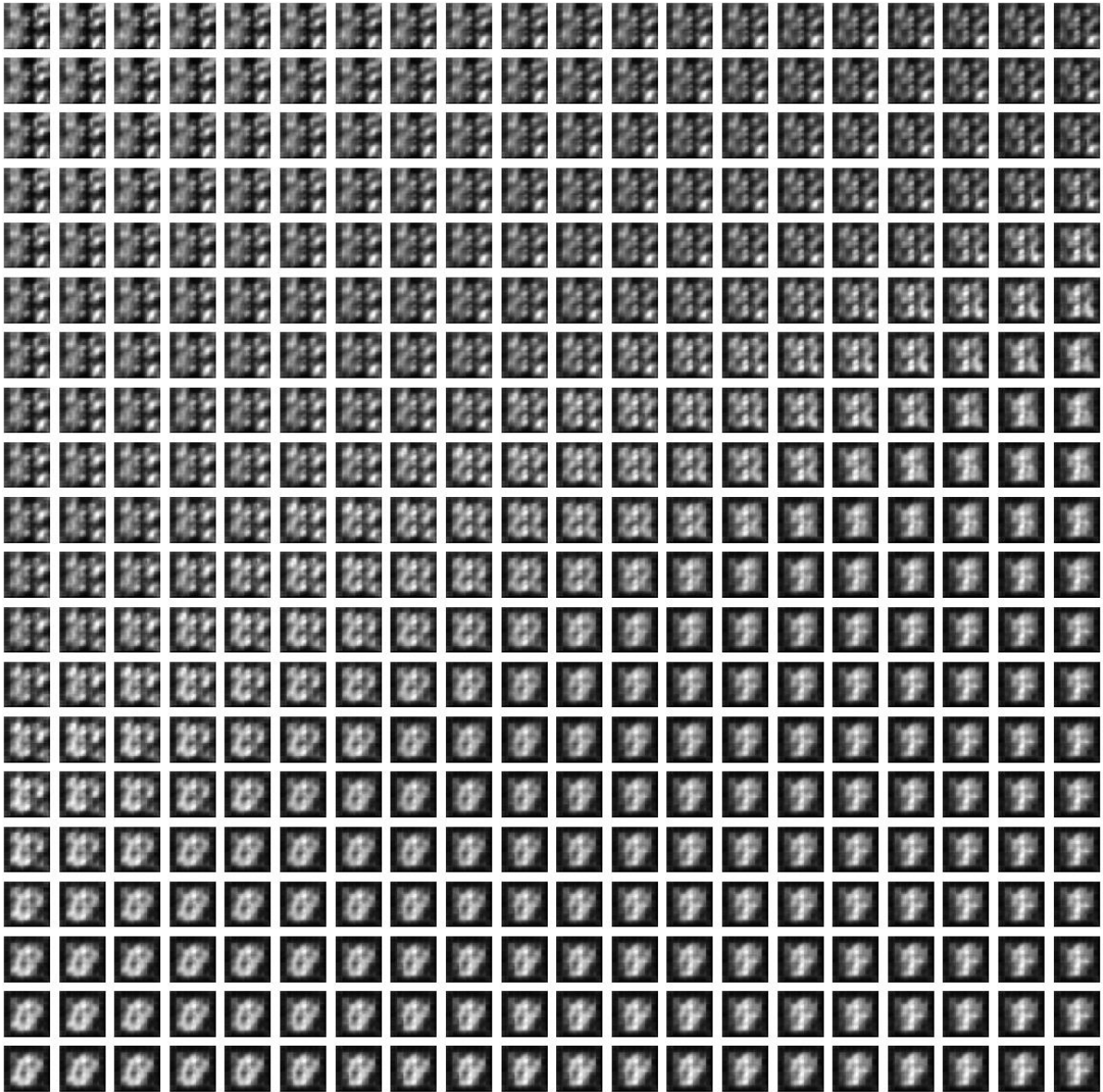


Figure 7. Latent space with added neurons architecture

In next step, I added both neurons and layers to the network, Fig (8) shows the losses for each phase. In fig (9), it is clear that the detection improved so much; however the latent space upside is still opaque, but some figures like 7, 4 and 9 can be seen in right half of the Fig (10).

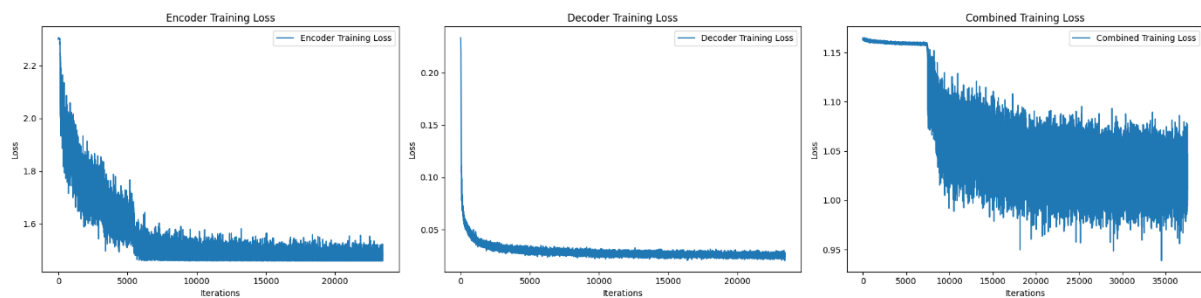


Figure 8. losses for each phase with added neurons and layers

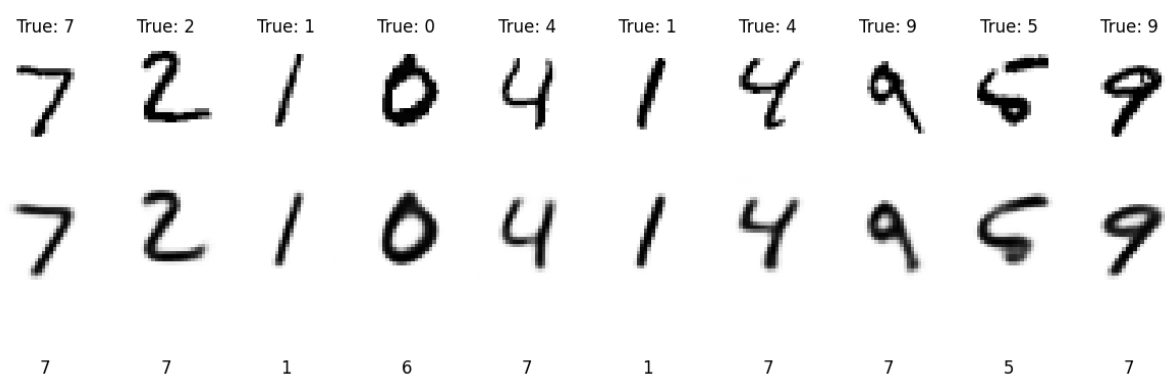


Figure 9. Test result with added neurons and layers

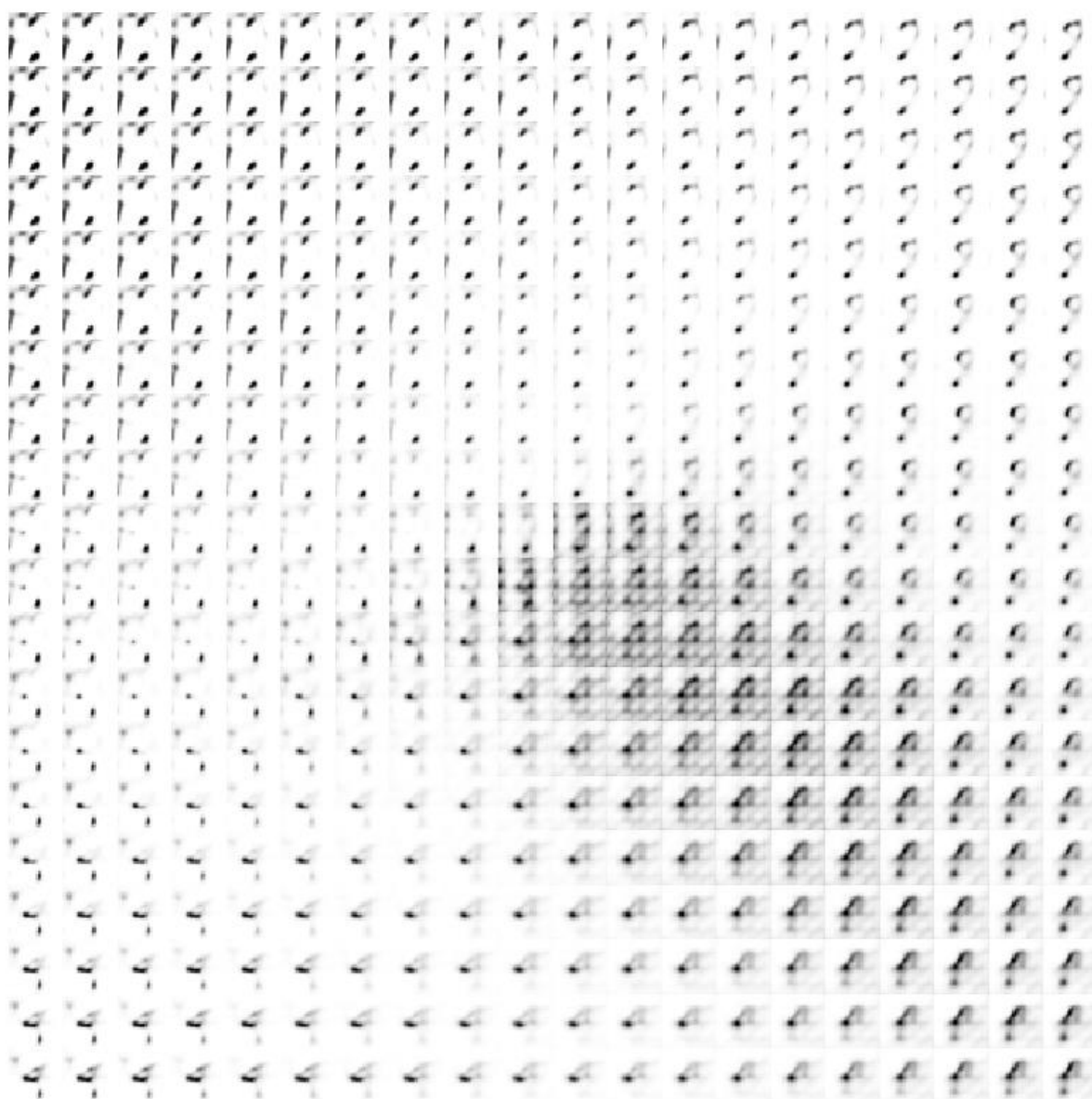


Figure 10. Latent space with added neurons and layers

In the next step I added one more layer with additive neurons to both encoder and decoder, Fig (11) shows the losses for each phase, the detection is the same as it is shown in Fig (12), but the latent space in Fig (13) seems to be more clear.

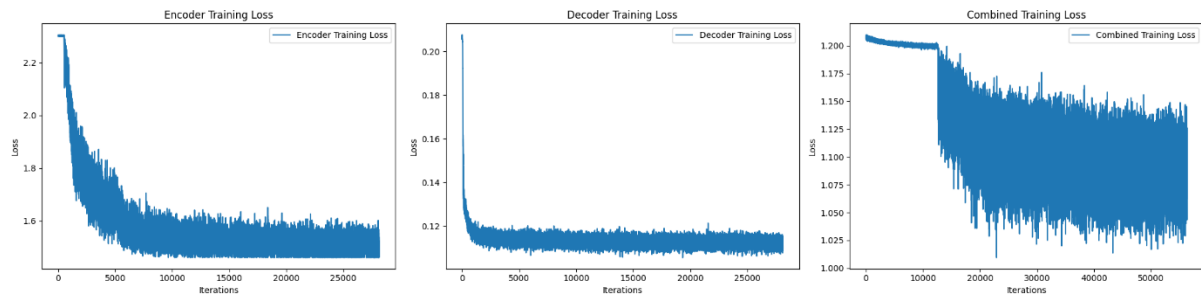


Figure 11. losses for each phase, added more neurons and layers

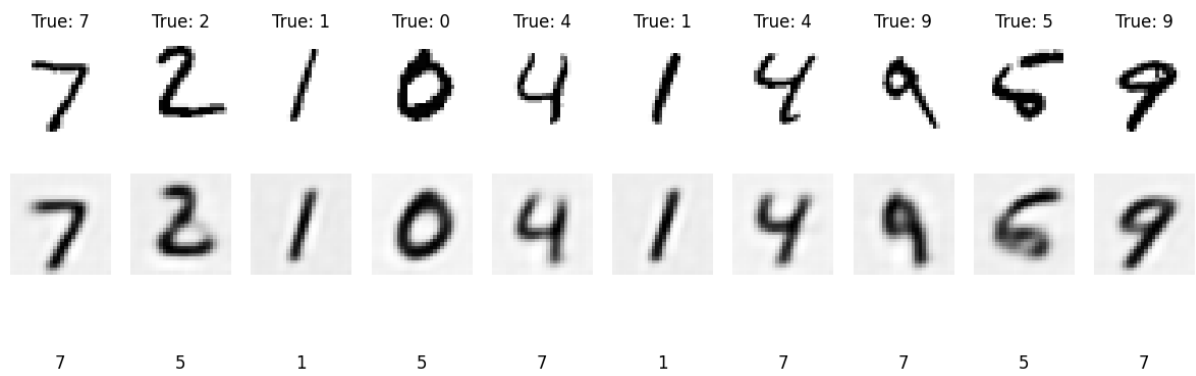


Figure 12. Test result with added more neurons and layers

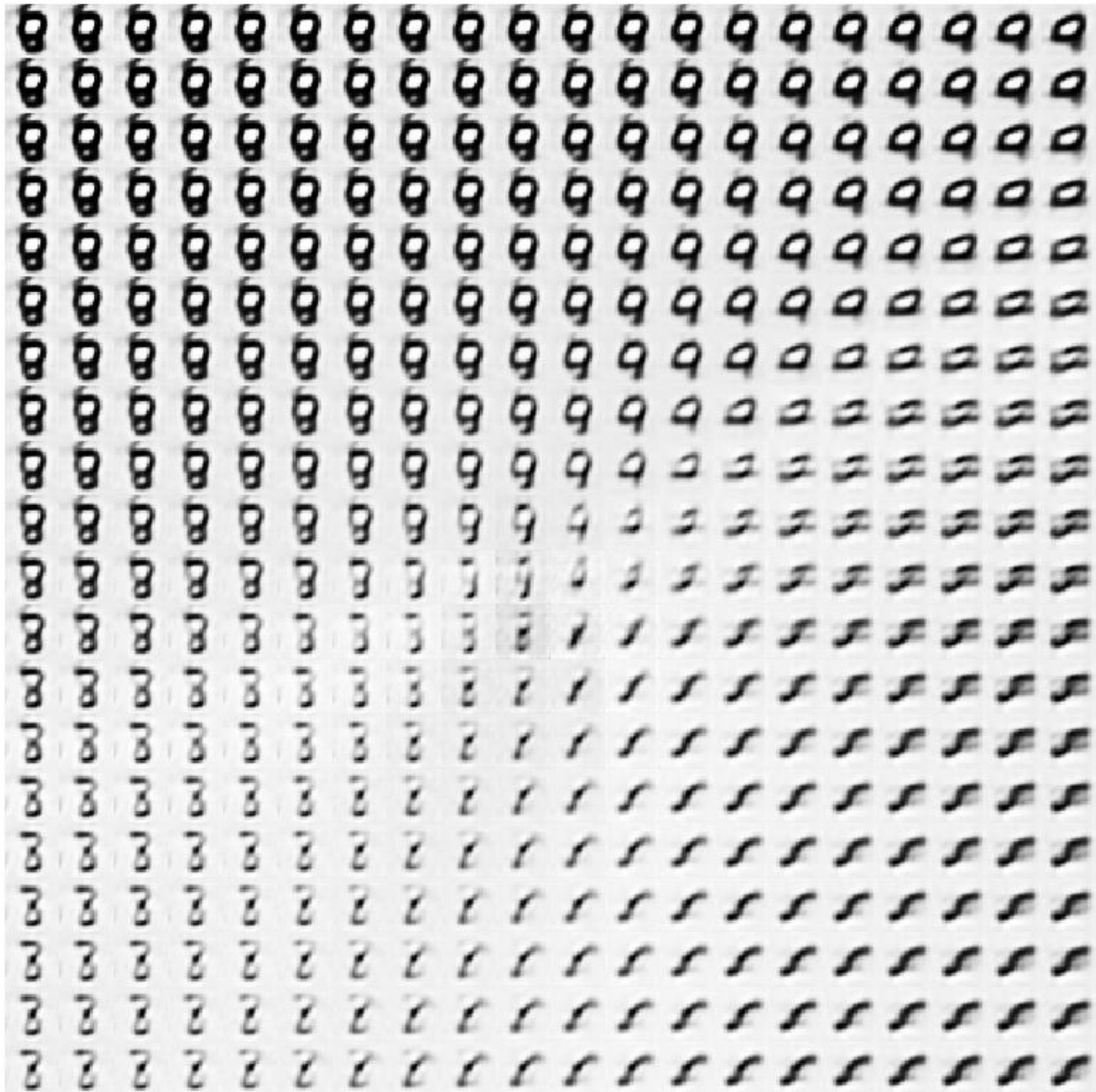


Figure 13. Latent space with added more neurons and layers

The result shows us that the denoising might not work well enough due to this configuration in the latent space of the decoder; however, the detection in the decoder with added noise is well enough.

2. Variational Auto Encoder

2. Repeat the above problem using Variational Autoencoder. In this case, the latent space should have two part: μ, σ . To do that, double the channels of last layer of the encoder to have $4 \times 1 \times 1$ latent space, in which the first two channels give a 2×1 vector μ , and the other two channels give a 2×1 vector σ . The Add a zero mean Gaussian noise to σ and then add that to μ . The result is 2×1 sampled output of encoder which will be the input of decoder. So decoder size will not change.

Visualize your result for the data and compare it with the result of the previous part.

2-1- Introduction

In this project, we developed a Variational Autoencoder (VAE) with a denoising capability using the MNIST dataset. The primary objective was to enhance the VAE's functionality to not only perform data reconstruction but also to effectively denoise the input images corrupted with salt-and-pepper noise. This report outlines the problem, methodology, results, and conclusions of our efforts.

2-2- Problem statement

The MNIST dataset consists of handwritten digits that are commonly used for training various image processing systems. For this project, we focused on:

1. Developing a VAE for the MNIST dataset.
2. Introducing salt-and-pepper noise to the input images to simulate real-world noise.
3. Training the VAE to denoise the images, aiming to reconstruct the original clean images from the noisy inputs.
4. Evaluating the VAE's performance in terms of denoising and classification accuracy.

2-3- Methodology

The MNIST dataset was preprocessed using standard normalization techniques. Additionally, we introduced a custom transformation to add salt-and-pepper noise to the images. This allowed us to have both clean and noisy versions of the dataset for training and evaluation.

- **Normalization:** Each image was normalized to have pixel values in the range [0, 1].
- **Salt-and-Pepper Noise:** A probability of 0.1 was used to corrupt the images, where 10% of the pixels were randomly set to either 0.1 (pepper) or 0.1 (salt).

2-4- Model Architecture

Several Model architecture have been applied to this implementation which will be discusses in results section; however, some other parameters will be explained here:

As it is assumed there are encode, decode and FC layer for classification. the problem specified to follow the instructions as below and the parameters are assumed as follow:

The training process involved multiple loss functions:

- **Reconstruction Loss:** Mean Squared Error (MSE) between the reconstructed images and the original clean images. This loss ensures that the output images are similar to the input images.
- **KL Divergence:** Measures the divergence between the learned latent space distribution and the standard normal distribution. This term regularizes the latent space to follow a Gaussian distribution.
- **Classification Loss:** Cross-entropy loss for digit classification. This loss ensures that the encoder can correctly classify the digits.

The total loss was a weighted sum of these components, allowing the model to learn both reconstruction and classification tasks.

Training was conducted in three distinct phases:

1. **Classification Phase:** Focused on training the classifier. During this phase, only the classifier parameters were updated.
 - **Optimizer:** Adam with learning rate 0.001
 - **Loss:** Classification loss (cross-entropy)
 - **Epochs:** 3
2. **Reconstruction Phase:** Focused on training the VAE to reconstruct clean images from noisy inputs. During this phase, only the reconstruction and KL divergence losses were considered.
 - **Optimizer:** Adam with learning rate 0.001
 - **Loss:** Reconstruction loss (MSE) + KL divergence
 - **Epochs:** 3
3. **Joint Training Phase:** Combined both classification and reconstruction objectives. During this phase, all parameters were updated based on a weighted sum of the classification and reconstruction losses.
 - **Optimizer:** Adam with learning rate 0.001
 - **Loss:** $0.5 * \text{Classification loss} + 0.5 * (\text{Reconstruction loss} + \text{KL divergence})$
 - **Epochs:** 4

2-5- Results

The python code in the zip file has been specified as Q2. First, I assumed there is no noise, and the basic architecture in the question has been employed to the implementation.

Fig (14) shows the losses for the specified parts in the training, the title of each plot has specified which one is related to which phase. Fig (15) shows the latent 2D latent space configuration as it has been asked in the question. It can be implied that the latent space is not seeing the numbers in a very semantic way with this architecture. Fig (16) shows the test data for the digits of the numbers, which seems to not to be very clear.

The reason of weak latent space Fig (15) and the detection might be the weakness of the architecture, lack of neurons, lack of convolutional layers, etc. I also changed the epochs and learning rate several times but no better output had come out.

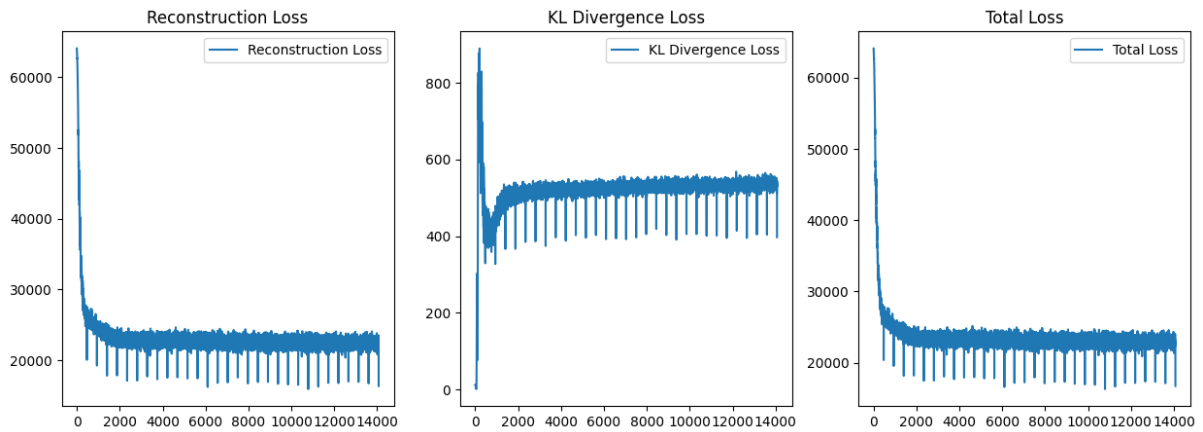


Figure 14. VAE losses for each phase, basic architecture

Latent Space Visualization

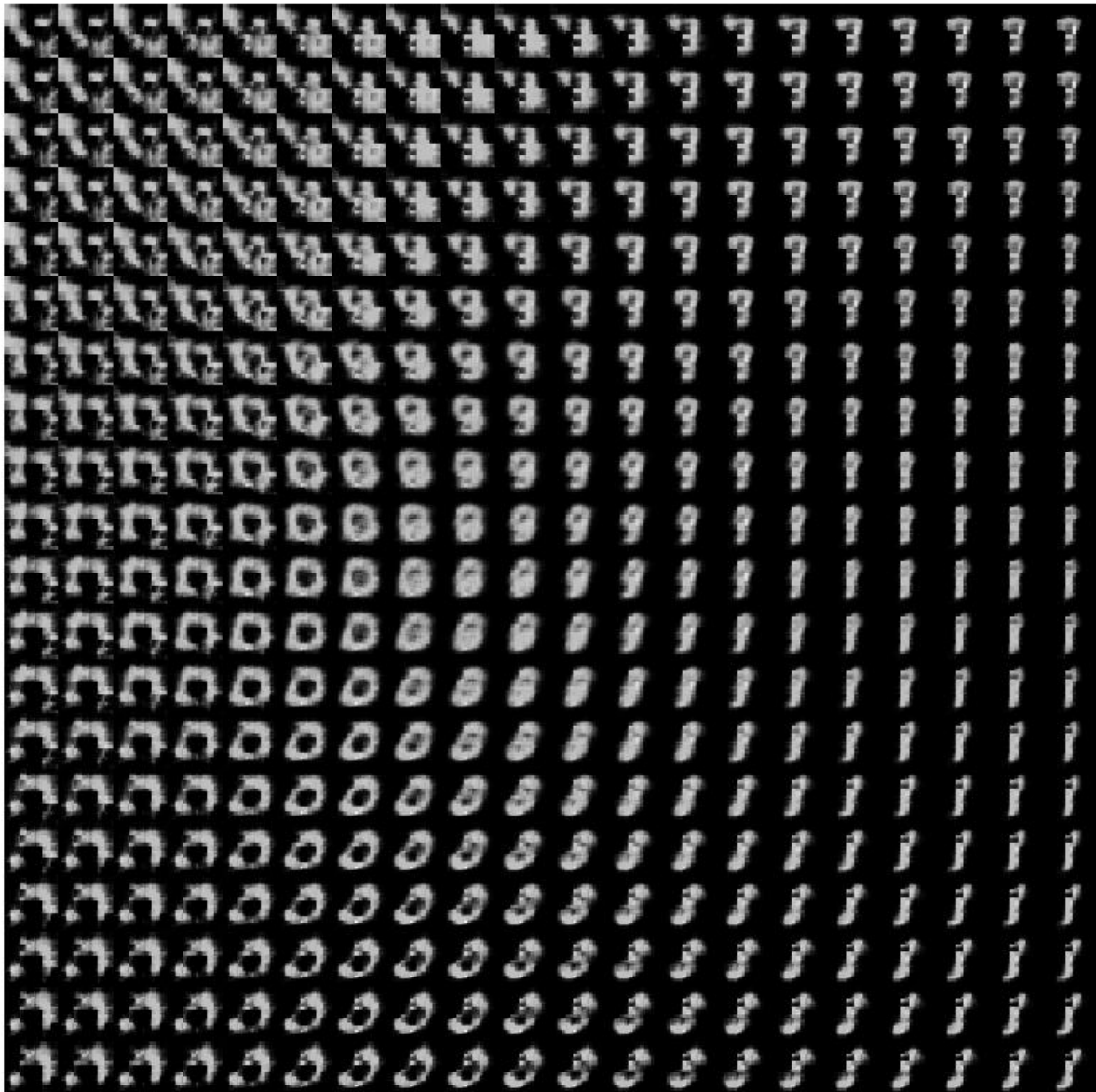


Figure 15. VAE latent space with basic architecture

Original and Reconstructed Images



Figure 16. VAE test result with basic architecture

Since the last part architecture did not work well enough to see latent space clearly, I added the to the neurons and layers in convolutional layers. The applied architecture can be shown as follows:

```
# Define the encoder
class Encoder(nn.Module):
    def __init__(self):
        super(Encoder, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=4, stride=2,
padding=1) # 28 -> 14
        self.dropout1 = nn.Dropout(0.1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2,
padding=1) # 14 -> 7
        self.dropout2 = nn.Dropout(0.1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=4, stride=2,
padding=1) # 7 -> 4
        self.dropout3 = nn.Dropout(0.1)
        self.conv4 = nn.Conv2d(128, 256, kernel_size=4, stride=1,
padding=1) # 4 -> 2
        self.dropout4 = nn.Dropout(0.1)
        self.fc_mu = nn.Linear(256 * 2 * 2, 2)
        self.fc_logvar = nn.Linear(256 * 2 * 2, 2)
        self.classifier = nn.Linear(2, 10) # Classifier for digit
classification

    def forward(self, x):
        x = torch.relu(self.conv1(x))
        x = self.dropout1(x)
        x = torch.relu(self.conv2(x))
        x = self.dropout2(x)
        x = torch.relu(self.conv3(x))
        x = self.dropout3(x)
        x = torch.relu(self.conv4(x))
        x = self.dropout4(x)
        x = x.view(x.size(0), -1)
        # print(f'Encoder output shape: {x.shape}')
        mu = self.fc_mu(x)
        logvar = self.fc_logvar(x)
        return mu, logvar, self.classifier(mu)

# Define the decoder
class Decoder(nn.Module):
    def __init__(self):
        super(Decoder, self).__init__()
        self.fc = nn.Linear(2, 256 * 2 * 2)
        self.deconv1 = nn.ConvTranspose2d(256, 128, kernel_size=4,
stride=2, padding=1) # 2 -> 4
```

```

        self.dropout1 = nn.Dropout(0.1)
        self.deconv2 = nn.ConvTranspose2d(128, 64, kernel_size=4,
stride=2, padding=1) # 4 -> 8
        self.dropout2 = nn.Dropout(0.1)
        self.deconv3 = nn.ConvTranspose2d(64, 32, kernel_size=4,
stride=2, padding=1) # 8 -> 16
        self.dropout3 = nn.Dropout(0.1)
        self.deconv4 = nn.ConvTranspose2d(32, 1, kernel_size=4,
stride=2, padding=1) # 16 -> 32

    def forward(self, z):
        z = self.fc(z)
        z = z.view(z.size(0), 256, 2, 2)
        z = torch.relu(self.deconv1(z))
        z = self.dropout1(z)
        z = torch.relu(self.deconv2(z))
        z = self.dropout2(z)
        z = torch.relu(self.deconv3(z))
        z = self.dropout3(z)
        z = torch.sigmoid(self.deconv4(z))
        z = z[:, :, 2:30, 2:30] # Crop to 28x28
        return z

# Define the VAE
class VAE(nn.Module):
    def __init__(self):
        super(VAE, self).__init__()
        self.encoder = Encoder()
        self.decoder = Decoder()

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        mu, logvar, _ = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        recon_x = self.decoder(z)
        return recon_x, mu, logvar

```

considering the above architecture, the losses over iterations has been shown in Fig (17). The latent space which is very clear has been shown in Fig (18), and finally, detection of the digits has been indicated in Fig (19). With adding neurons and layers all parts has been significantly improved.

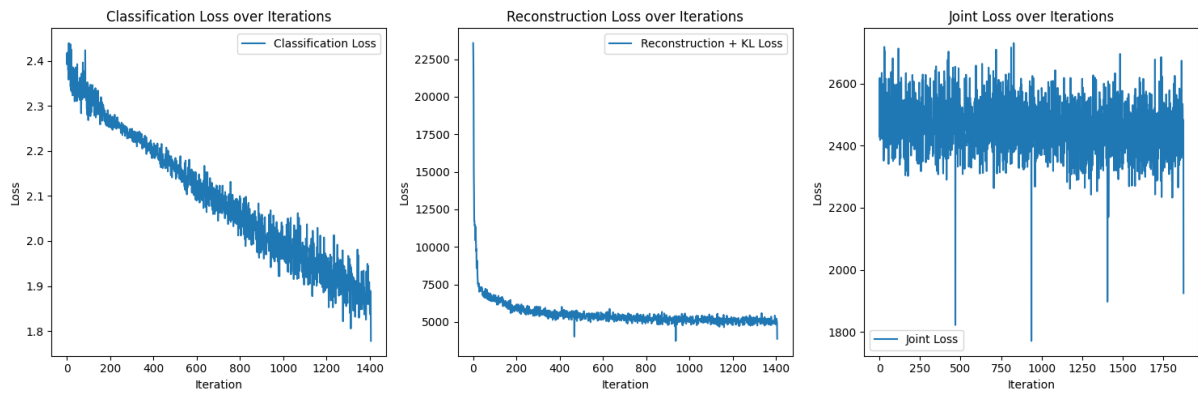


Figure 17. VAE losses for each phase, added neurons

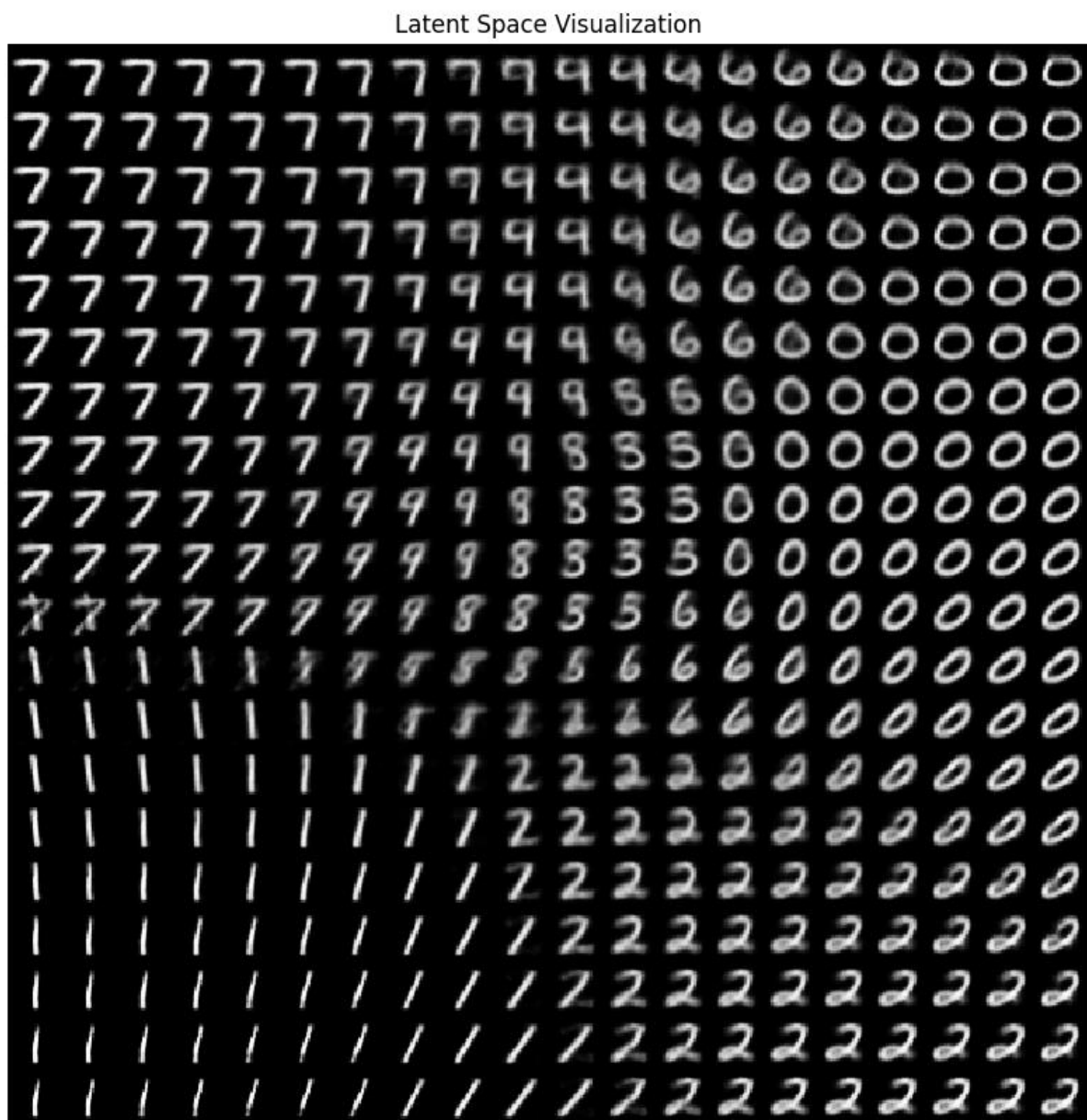


Figure 18. VAE latent space, added neurons

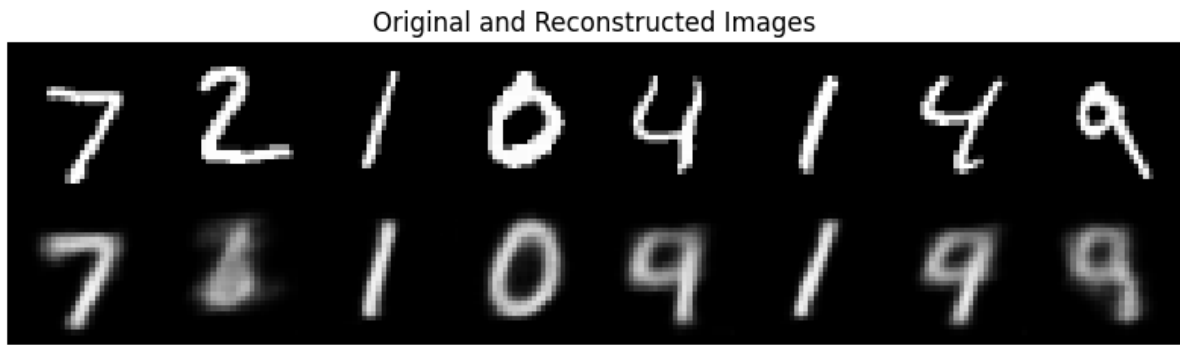


Figure 19. VAE test data, added neurons

In this part, I added salt and pepper noise to the configuration, and implemented the same network with considering the denoising, comparing the decode layer loss with actual images. However, it must be considered what is being shown in Fig(22) indicates how the network can detect compared to the noisy data, it is clear that the last digit prevailed with lots of noise both pepper and salt, and the network could not overcome that much noise. It is also clear in the Fig (21) that the latent space could not see this digit very clear and there is almost nothing to compare with real data. Fig (20) shows losses over iterations.

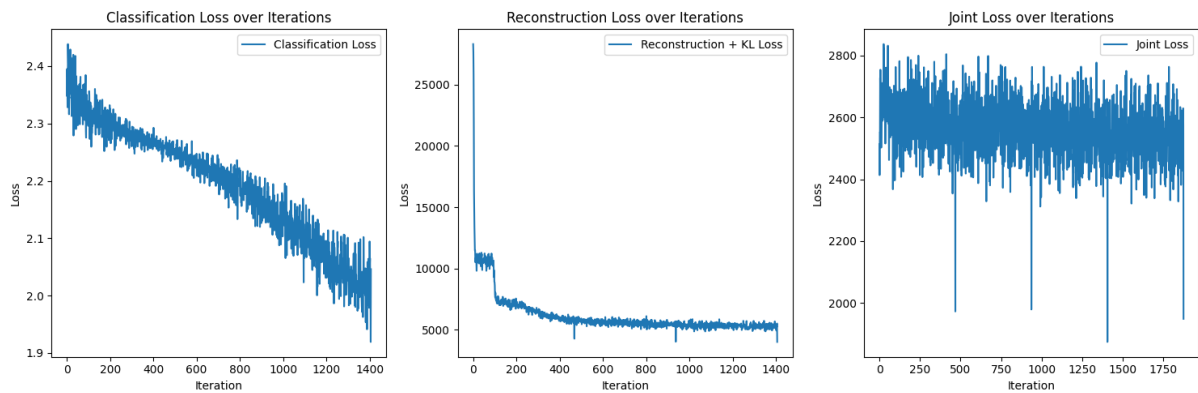


Figure 20. VAE losses for each phase, added neurons with noise

Latent Space Visualization

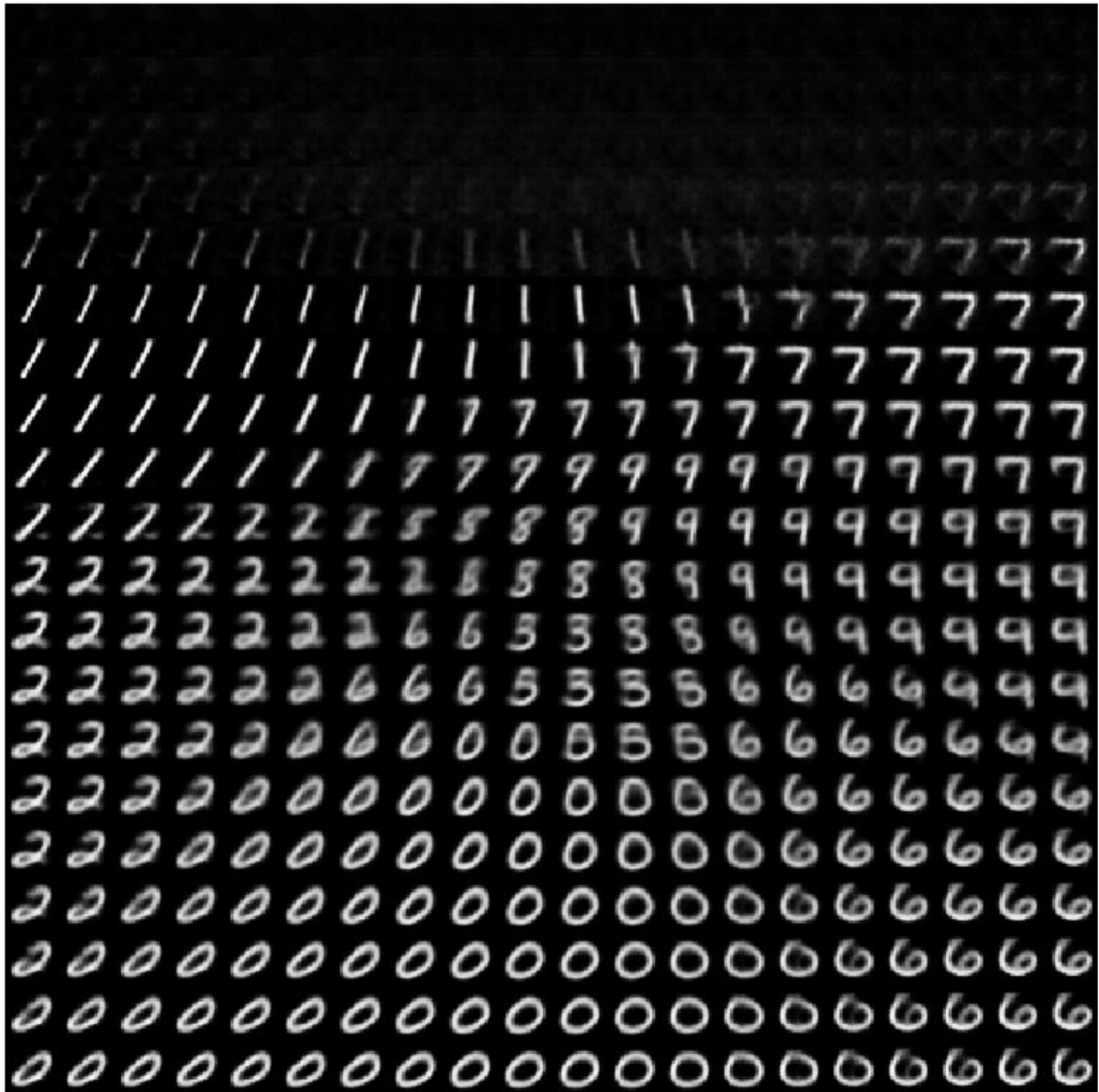


Figure 21. VAE latent space, added neurons and noise

Original and Reconstructed Images

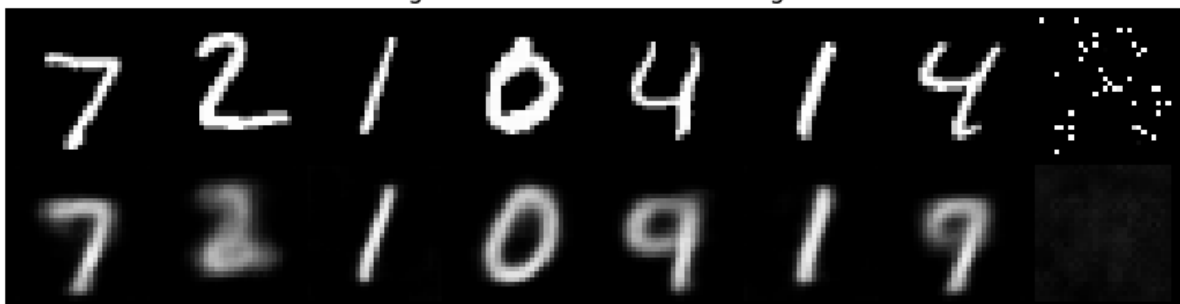


Figure 22. VAE test data, added neurons and noise

In next step, I added some more layers and neurons to the encode, decode and FC. Fig (23) shows the loss for each phase. The latent space visulization in Fig (24) shows that one figure is realy noisy; the Fig (25) shows that the second didgit had lots of noise which could not be detected at all, but other figures even with noise had been detected as they are sort of clear in latent space,too.

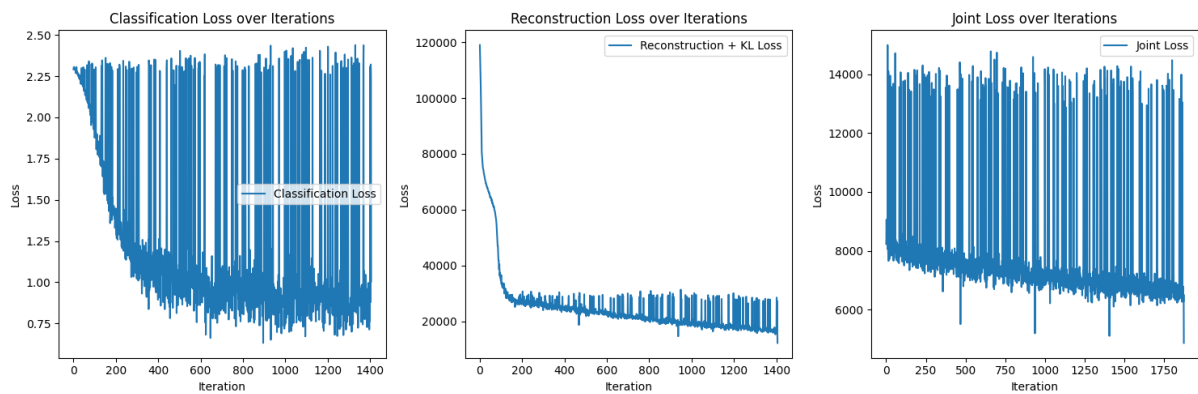


Figure 23. VAE losses for each phase, added neurons and layers with noise

Latent Space Visualization

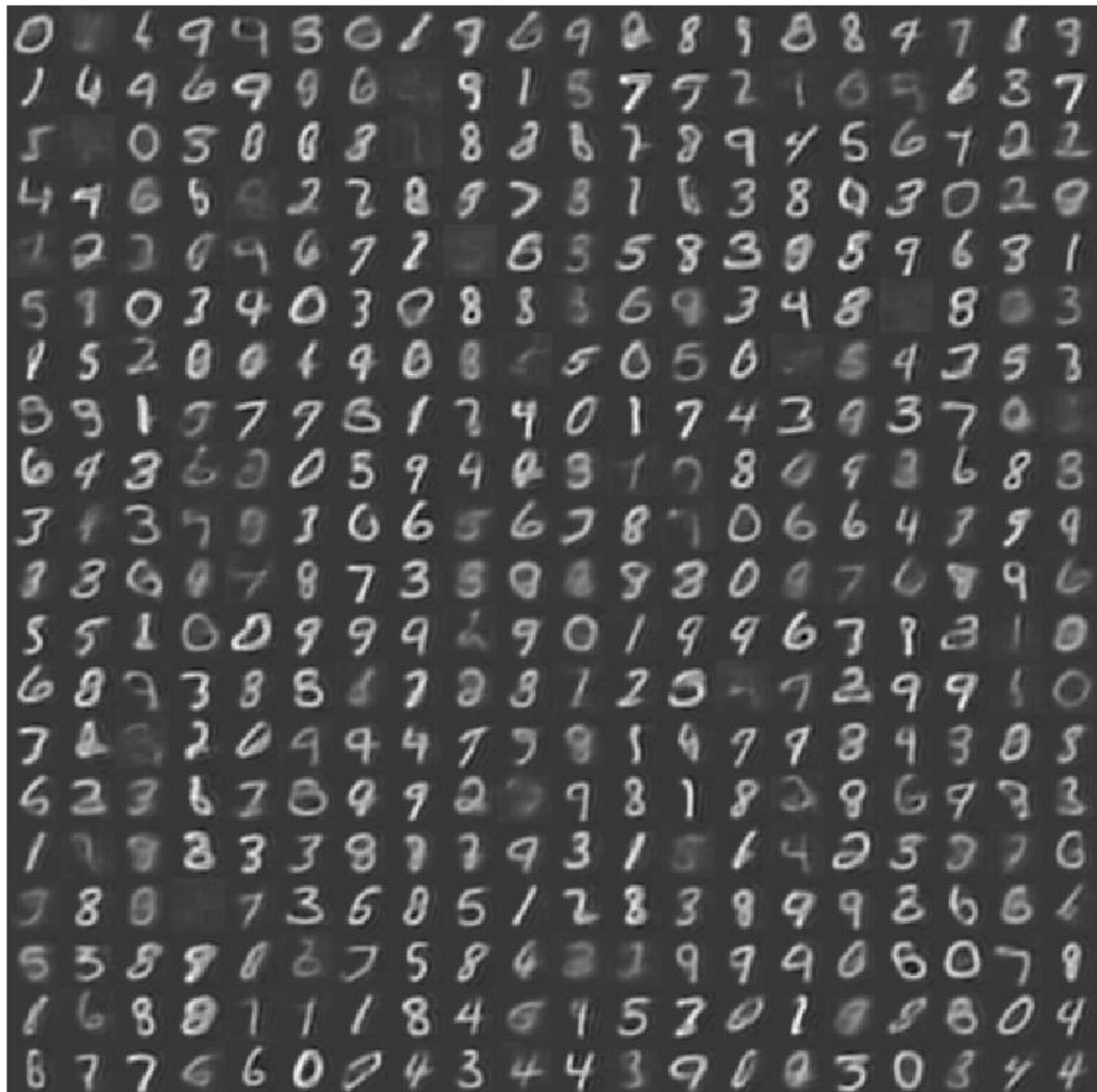


Figure 24. VAE latent space, added neurons, layers and noise

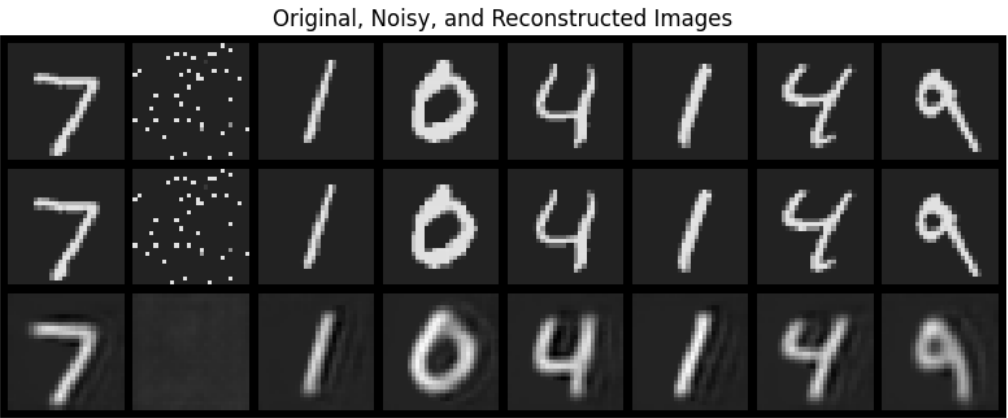


Figure 25. VAE test data, added neurons, layers and noise